In [7]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

In [8]:
```
from sklearn.datasets import fetch_california_housing
import pandas as pd

housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["HousePrice"] = housing.target

df.head()
```

Out[8]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latit
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	3
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	3
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	3
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	3
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	3

In [9]:
```
print("Rows and Columns:", df.shape)
```

```
Rows and Columns: (20640, 9)
```

In [10]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   MedInc      20640 non-null  float64
 1   HouseAge    20640 non-null  float64
 2   AveRooms    20640 non-null  float64
 3   AveBedrms   20640 non-null  float64
 4   Population  20640 non-null  float64
 5   AveOccup    20640 non-null  float64
 6   Latitude    20640 non-null  float64
 7   Longitude   20640 non-null  float64
 8   HousePrice  20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

In [11]:
```
df.describe()
```

Out[11]:

	MedInc	HouseAge	AveRooms	AveBedrms	Populatio
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.00000
mean	3.870671	28.639486	5.429000	1.096675	1425.47674
std	1.899822	12.585558	2.474173	0.473911	1132.46212
min	0.499900	1.000000	0.846154	0.333333	3.00000
25%	2.563400	18.000000	4.440716	1.006079	787.00000
50%	3.534800	29.000000	5.229129	1.048780	1166.00000
75%	4.743250	37.000000	6.052381	1.099526	1725.00000
max	15.000100	52.000000	141.909091	34.066667	35682.00000

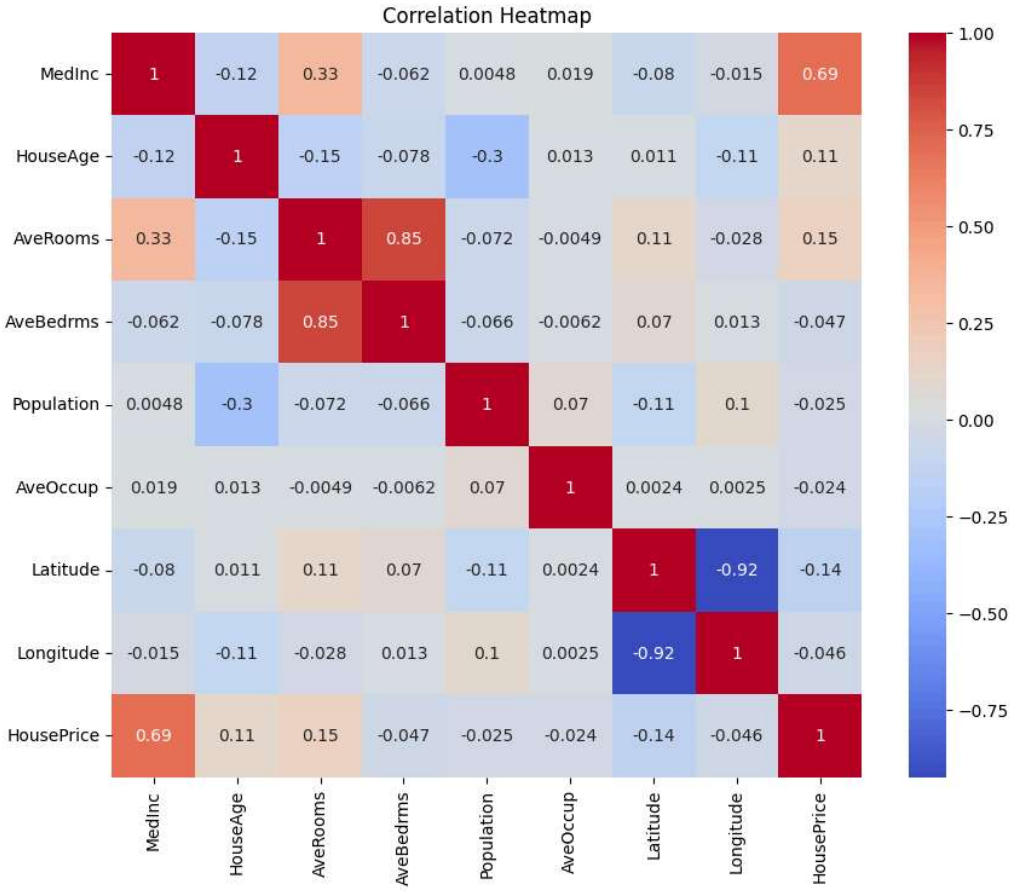
In [12]:
```
df.isnull().sum()
```

Out[12]:
```
MedInc        0
HouseAge      0
AveRooms      0
AveBedrms     0
Population    0
AveOccup      0
Latitude      0
Longitude     0
HousePrice    0
dtype: int64
```

In [13]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [14]:
```
plt.figure(figsize=(8,5))
sns.histplot(df["HousePrice"], bins=30, kde=True)
plt.title("House Price Distribution")
plt.show()
```

In [15]:
```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



In [16]:
```
X = df.drop("HousePrice", axis=1)
y = df["HousePrice"]
```

In [17]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (16512, 8)
Testing data: (4128, 8)
```

In [18]:
```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Model trained successfully!")
```

```
Model trained successfully!
```

In [19]:
```
y_pred = model.predict(X_test)

print(y_pred[:10])
```

```
[0.71912284 1.76401657 2.70965883 2.83892593 2.60465725 2.01175367
 2.64550005 2.16875532 2.74074644 3.91561473]
```

In [20]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)
```

```
Mean Absolute Error (MAE): 0.5332001304956555
Root Mean Squared Error (RMSE): 0.7455813830127763
R2 Score: 0.575787706032451
```

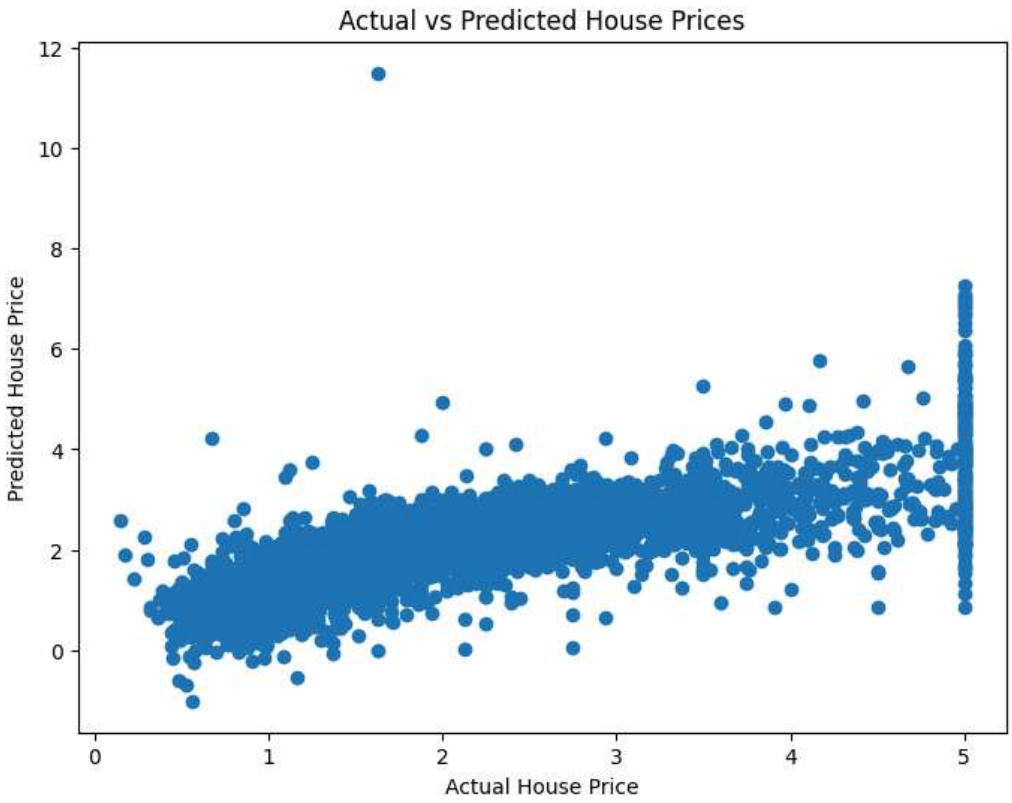
In [21]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted House Prices")

plt.show()
```



In [22]:
```
import pickle

with open("house_price_model.pkl", "wb") as file:
    pickle.dump(model, file)

print("Model saved successfully!")
```

```
Model saved successfully!
```

House Price Prediction using Linear Regression

Internship Task 1

Name: Aariya

Model: Linear Regression

Dataset: California Housing Dataset

Objective

The objective of this project is to build a Linear Regression model to predict house prices using the California Housing dataset. The project demonstrates data loading, preprocessing, exploratory data analysis, model training, prediction, and performance evaluation.

Dataset Description

The California Housing dataset contains information about houses in California, including median income, house age, average rooms, average bedrooms, population, average occupancy, latitude, and longitude.

Target Variable:

- HousePrice

Exploratory Data Analysis

- Checked dataset shape
- Displayed dataset information
- Generated statistical summary
- Checked for missing values
- Visualized house price distribution
- Generated correlation heatmap

Model Training

The dataset was divided into training and testing sets using train_test_split. A Linear Regression model from scikit-learn was trained using the training data.

Model Evaluation

The model performance was evaluated using:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Results

MAE = 0.5332001304956555

RMSE = 0.7455813830127763

R^2 Score = 0.575787706032451

Conclusion

The Linear Regression model was successfully trained on the California Housing dataset and evaluated using standard regression metrics. The project demonstrates the complete machine learning workflow from data loading to prediction and evaluation.